

BCS501(SEPM) SIMP Questions

-RNSIT REVIEW TEAM.

SIMP Questions M1-M5

Note: Software engineering is a theory intensive subject which almost involves theory intensive and a lot of important topics

Understanding how these processes work and writing answers on your own is the key to scoring marks in this particular subject

Understand every question by referring to its answer on chatgpt or Online or TIE Notes by considering a real life example ,Study 75% of these questions- These questions are framed by Professors from over 5 Colleges and have been combined to give you the most important topics

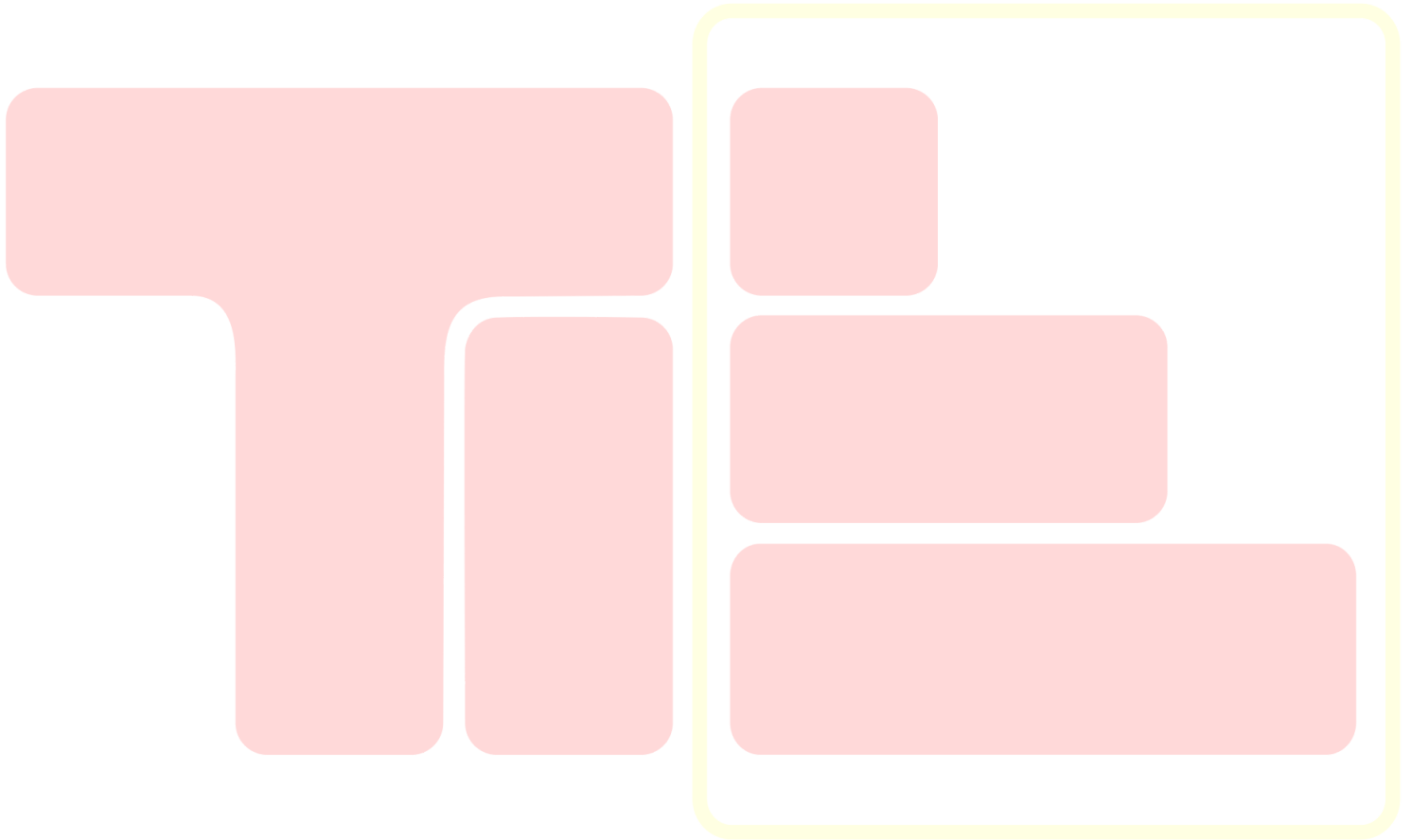
There are approx 8 questions per module, you can practice any 6/M

1. Explain the fundamental activities of generic software processes and explain generic process model
2. Explain the seven principles of software engineering practice proposed by david hooker
3. Explain the unique nature of web applications wrt development
4. Define process patterns? Explain the template for describing process patterns proposed by ambler
5. Explain component based software development and list the different steps that are undertaken during component based development
6. Define software as per fritz baur and IEEE? Explain the characteristics of software that are considerably different from those of hardware
7. Write a short note on spiral and concurrent model
8. Explain waterfall model, list the disadvantages of the same
9. Explain the different phases of unified process model
10. Explain the steps required to establish the groundwork for an understanding of software requirements

11. Explain requirement engineering wrt Inception, elicitation, Elaboration , Negotiation and Validation- Explain how negotiation and validating requirements works in detail-SRS(4+6)
12. Explain the process of requirement elicitation and the approaches involved with it
13. Explain requirement engineering? Explain briefly the seven tasks of requirement engineering
14. Explain different types of data modeling concepts with example
15. Explain the development of use-case with a diagram and mention its advantages?
16. Describe the following with Example a. Class Diagram b. Generalization
17. Draw a Sequence and UML diagram for SafeHome security function.
18. Draw an activity diagram for an ATM.
19. What is an Agile process? Write the principles to achieve agility.
20. Explain the extreme programming process (XP) with a neat diagram.
21. Explain Scrum process flow with a neat diagram.
22. What are the templates for defining a feature and explain the feature driven development process with a neat diagram.
23. Write short notes on (i) Principle that guides each frame and activity (ii) RDMP
24. Discuss the prototyping model. What is the effect of designing a prototype on the overall cost of the project?
25. Differentiate between top down and bottom-up design strategies.
26. Define a project? Explain the characteristics of projects.
27. What are the types of requirements? Explain ISO 12207 software development life cycle with a neat diagram.
28. Explain the functions of the object driven development process.
29. Discuss Test Driven Development(TDD) with its Process and list its benefits.
30. What is management? Explain the several key activities of software project management.
31. Discuss the importance of project planning. Show the principles of project management processes, With neat diagram explaining the project scheduling process.
32. Explain the project control cycle with a neat diagram.
33. Discuss software Quality and its attributes, explain process based quality of software.
34. Explain Software Evolution Process with neat block diagram
35. What are the key points of step wise framework explained with a neat diagram.
36. Briefly explain the product and process metrics.
37. Explain

test closure and test automation with a neat diagram. 38. Explain the different SEI CMM levels.

39. Briefly explain ISO 9001:2000 process with its principles.



Module -1 Scheme of Evaluation-TIE

1. Fundamental Activities of Generic Software Processes and Generic Process Model

Fundamental Activities:

1. Specification: Defining what the system should do.
2. Design and Implementation: Creating the software architecture and writing the code.
3. Validation: Ensuring the software meets the requirements.
4. Evolution: Modifying the software to meet changing needs.

Generic Process Model:

1. Waterfall Model: Linear and sequential approach.
2. Incremental Model: Develops the system in increments.
3. Iterative Model: Develops the system through repeated cycles (iterations).
4. Spiral Model: Combines iterative development with systematic aspects of the waterfall model, focusing on risk assessment.
5. Agile Methods: Emphasizes flexibility, collaboration, and customer feedback.

2. Seven Principles of Software Engineering Practice (David Hooker)

1. The Reason It All Exists: The primary goal of software is to provide value to its users.
2. Keep It Simple, Stupid (KISS): Simplicity should be a key goal in design and implementation.
3. Maintain the Vision: Have a clear vision and goal for the project.
4. What You Produce, Others Will Consume: Code should be readable and maintainable for others.
5. Be Open to the Future: Design with potential future changes in mind.
6. Plan Ahead for Reuse: Create software components that can be reused in future projects.

7. Think!: Constantly think about what you are doing and why.

3. Unique Nature of Web Applications with Respect to Development

- User Interface: Web applications require dynamic, user-friendly interfaces.
- Performance: Must handle multiple users and scale effectively.
- Security: Web applications are vulnerable to a variety of security threats.
- Integration: Often need to integrate with other web services and databases.
- Deployment: Continuous deployment and integration are crucial for web apps.
- Diverse User Base: Must support various devices, browsers, and platforms.

4a. Process Patterns and Template for Describing Process Patterns (Ambler)

Process Patterns:

Reusable solutions to common problems in software development.

Template for Describing Process Patterns (Ambler):

- Pattern Name: The name of the pattern.
- Intent: The goal the pattern aims to achieve.
- Motivation: The context or problem the pattern addresses.
- Applicability: Situations where the pattern is applicable.
- Structure: A detailed description of the pattern's structure.
- Participants: The roles involved in the pattern.
- Collaborations: How participants interact.
- Consequences: The results and trade-offs of using the pattern.
- Implementation: Guidelines for implementing the pattern.
- Sample Code: Code examples illustrating the pattern.
- Known Uses: Examples of real-world use.

- Related Patterns: Other patterns that are related.

4b. Activities Involved in Software Development Framework

1. Requirement Analysis: Gathering and analyzing requirements.
2. System Design: Creating the architecture of the system.
3. Implementation: Writing and compiling the code.
4. Testing: Verifying the software against requirements.
5. Deployment: Delivering the system to users.
6. Maintenance: Ongoing support and updates.

5. Component-Based Software Development (CBSD)

Definition: An approach that emphasizes the design and construction of computer-based systems using reusable software components.

Steps in Component-Based Development:

1. Requirement Analysis: Identify the system's requirements.
2. Component Qualification: Evaluate and select existing components.
3. Component Adaptation: Modify components to fit the system.
4. System Design: Integrate components into the system architecture.
5. Implementation: Assemble the components into a working system.
6. Testing: Ensure the system works as intended.
7. Maintenance: Update components as requirements change.

6. Definition of Software (Fritz Bauer and IEEE)

Fritz Bauer: "Software is the collection of programs, procedures, rules, and associated documentation and data."

IEEE: "Software is a collection of computer programs, procedures, rules, and associated documentation and data pertaining to the operation of a computer system."

Characteristics of Software vs. Hardware:

- Intangibility: Software is intangible; hardware is tangible.
- Flexibility: Software can be changed more easily than hardware.
- Reproducibility: Software can be duplicated without cost; hardware requires physical materials.
- Dependency on Execution: Software performance depends on hardware execution.
- Aging: Software doesn't wear out but can become outdated; hardware deteriorates over time.

7. Spiral and Concurrent Models

Spiral Model:

- Combines iterative development with systematic aspects.
- Focuses on risk assessment.
- Iterations called "spirals."
- Each spiral includes planning, risk analysis, engineering, and evaluation.

Concurrent Model:

- Activities occur simultaneously.
- Software development is seen as a network of activities.
- Emphasizes parallelism and concurrency.

8. Waterfall Model

Phases:

1. Requirement Analysis: Gathering requirements.
2. System Design: Planning the architecture.
3. Implementation: Writing code.
4. Integration and Testing: Combining and testing components.
5. Deployment: Delivering the product to users.
6. Maintenance: Ongoing support and updates.

Disadvantages:

- Inflexibility: Difficult to accommodate changes.
- Late Testing: Issues are found late in the process.
- Limited User Involvement: Users are only involved at the beginning and end.

9. Phases of Unified Process Model

1. Inception: Define the scope and purpose of the project.
2. Elaboration: Analyze the problem domain, establish an architectural foundation, and eliminate high-risk elements.
3. Construction: Build the software product.
4. Transition: Move the system to the user community.
5. Production: Maintain and support the system.

These detailed explanations should provide a comprehensive understanding of each topic in software engineering.

Module 2 to Module 5

Scheme of Evaluation or Key Answers for BCS501 SIMP - by TIE TEAM

10. Steps to Establish the Groundwork for Understanding Software Requirements

To establish the groundwork for understanding software requirements, several critical steps must be followed:

1. Stakeholder Identification:

- Identify all stakeholders, including clients, end-users, and other parties affected by the software. Understanding their needs and expectations is crucial.

2. Requirement Gathering:

- Collect initial requirements from stakeholders through interviews, surveys, and discussions. Focus on understanding their needs and business goals.

3. Establish Objectives:

- Define the primary objectives of the software project. What problem is it solving? What are the success criteria?

4. Scope Definition:

- Clearly outline the scope of the project. Define what is within the scope and what is outside of it to manage expectations.

5. Feasibility Study:

- Conduct a feasibility study to determine whether the project is technically and economically viable.

6. Requirement Prioritization:

- Prioritize the requirements based on stakeholder needs, project goals, and resource availability.

7. Documentation:

- Document all gathered requirements in a structured format. This may include a Software Requirements Specification (SRS) document.

8. Validation with Stakeholders:

- Review the documented requirements with stakeholders to ensure accuracy and completeness.

9. Create a Requirement Traceability Matrix (RTM):

- Develop an RTM to track requirements throughout the project lifecycle.

10. Baseline the Requirements:

- Once agreed upon, baseline the requirements to serve as a reference point for all future development activities.

11. Requirement Engineering: Inception, Elicitation, Elaboration, Negotiation, and Validation

Requirement Engineering is a systematic approach to defining, documenting, and maintaining software requirements. It involves several phases:

1. Inception:

- The inception phase involves defining the initial scope and understanding the basic requirements. Stakeholders are identified, and the overall goals of the project are discussed.

2. Elicitation:

- Elicitation is the process of gathering requirements from stakeholders through interviews, workshops, surveys, and other techniques. The goal is to understand what the stakeholders need from the system.

3. Elaboration:

- Elaboration involves refining and expanding the requirements gathered during the elicitation phase. Detailed use cases, models, and scenarios are developed to provide a deeper understanding of the requirements.

4. Negotiation:

- During negotiation, stakeholders prioritize requirements and resolve conflicts. This step ensures that all stakeholders agree on the final set of requirements, balancing their needs and constraints.

5. Validation:

- Validation involves ensuring that the documented requirements accurately reflect the stakeholder's needs and are feasible within the project's constraints. Techniques like reviews, prototypes, and test cases are used to validate the requirements.

Negotiation and Validation in Detail:

- Negotiation:

- Stakeholders may have conflicting requirements, so negotiation helps to reach a consensus. Techniques like prioritization (e.g., MoSCoW method: Must have, Should have, Could have, Won't have) are used. The goal is to achieve a mutually agreed set of requirements that align with project constraints such as budget, time, and resources.

- Validation:

- Validation ensures that the requirements are correct, complete, and aligned with stakeholder expectations. Methods like reviews, inspections, walkthroughs, and prototyping are used. Validation checks are done against the Software Requirements Specification (SRS) to confirm that the final product will meet the stakeholders' needs.

12. Requirement Elicitation Process and Approaches

Requirement Elicitation is the process of gathering requirements from stakeholders. The process involves several steps:

1. Preparation:

- Understand the problem domain, identify stakeholders, and prepare materials for elicitation.

2. Conducting Elicitation:

- Use various techniques to gather requirements:
 - Interviews: Direct discussions with stakeholders.
 - Workshops: Collaborative sessions with stakeholders.
 - Surveys/Questionnaires: Collecting information from a large group.
 - Observation: Watching end-users interact with the current system.
 - Prototyping: Creating a preliminary model to gather feedback.

3. Documenting Requirements:

- Record the gathered information in a structured format, such as use cases or user stories.

4. Verification and Validation:

- Ensure that the gathered requirements are accurate and reflect stakeholder needs.

5. Prioritization:

- Rank the requirements based on their importance and impact on the project.

6. Final Review:

- Review the documented requirements with stakeholders for final approval.

Approaches Involved in Requirement Elicitation:

- Traditional Approaches:

- Interviews: Structured or unstructured conversations with stakeholders to gather detailed information.

- Surveys: Distributing questionnaires to gather information from a large audience.

- Document Analysis: Reviewing existing documentation to understand the current system.

- Collaborative Approaches:

- Workshops: Group sessions where stakeholders collaboratively discuss and define requirements.

- Brainstorming: Generating ideas in a group setting, focusing on quantity before quality.

- Contextual Approaches:

- Observation: Watching users interact with the system to understand their needs and challenges.

- Prototyping: Developing a preliminary version of the system to gather feedback.

13. Requirement Engineering and Its Seven Tasks

Requirement Engineering is the process of defining, documenting, and maintaining software requirements. The seven tasks of requirement engineering include:

1. Inception:

- Establish the project's scope and identify stakeholders. Initial discussions help in understanding the overall goals.

2. Elicitation:

- Gathering requirements from stakeholders through various techniques like interviews, workshops, and observation.

3. Elaboration:

- Refining and expanding the requirements gathered during elicitation. Develop use cases and models to understand the system better.

4. Negotiation:

- Resolving conflicts and prioritizing requirements. Stakeholders agree on a final set of requirements.

5. Specification:

- Documenting the requirements in a clear, concise, and consistent manner. This may include use cases, user stories, or a Software Requirements Specification (SRS).

6. Validation:

- Ensuring that the requirements accurately reflect stakeholder needs. Techniques like reviews, prototypes, and test cases are used.

7. Management:

- Managing changes to the requirements over the course of the project. This includes tracking changes, version control, and impact analysis.

14. Different Types of Data Modelling Concepts with Examples

Data Modelling is the process of creating a data model to represent data structures and relationships. Key concepts include:

1. Entity:

- Represents a real-world object or concept. For example, in a school database, "Student" can be an entity.

2. Attribute:

- Describes properties of an entity. For example, a "Student" entity might have attributes like "Student_ID", "Name", and "Date_of_Birth".

3. Relationship:

- Describes how entities are related. For example, a "Student" entity might have a relationship with a "Domain" entity, indicating that students enroll in TIE Internships.

4. Primary Key:

- A unique identifier for an entity. For example, "Student_ID" could be the primary key for the "Student" entity.

5. Foreign Key:

- An attribute that creates a link between two entities. For example, "Domain ID" in the "Enrollment" entity could be a foreign key linking to the "Domain" entity.

6. Normalization:

- The process of organizing data to reduce redundancy. For example, breaking down a "Student" table into "Student" and "Address" tables.

Example:

For a TIE Internship database with companies:

- Entities: Student, Domain, Instructor

- Attributes: Student_ID, Name, Domain_Name, Instructor_Name
- Relationships: Students enroll in Various domains such as full stack development; These are taught by Instructors.

15. Development of Use-Case with Diagram and Its Advantages

A Use-Case is a detailed description of how users interact with a system to achieve a specific goal.

Steps to Develop a Use-Case:

1. Identify Actors:

- Identify who will interact with the system (e.g., users, other systems).

2. Define Goals:

- Determine what the actors want to achieve.

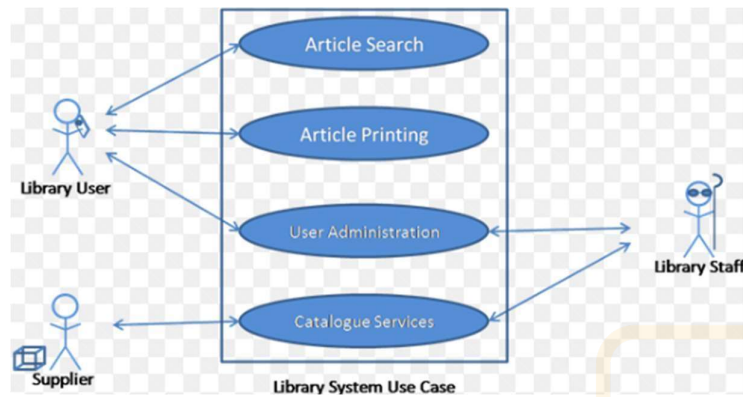
3. Outline Use-Case Scenarios:

- Describe the steps the actors will take to achieve their goals.

4. Create a Use-Case Diagram:

- Visual representation of the interactions between actors and the system.

Example Use-Case Diagram:



Advantages:

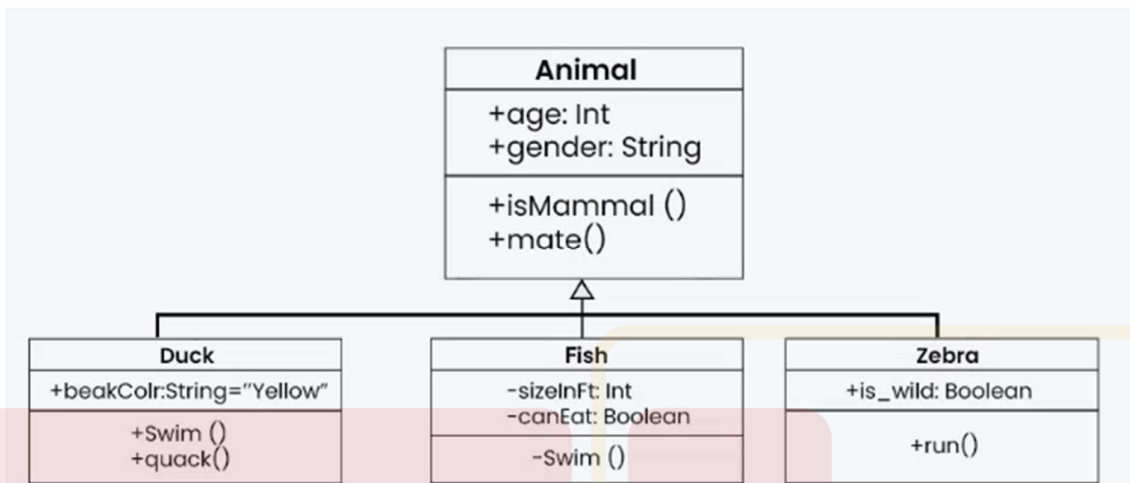
- Clarity: Provides a clear and concise way to capture functional requirements.
- Communication: Facilitates communication between stakeholders and developers.
- Traceability: Helps in tracing requirements through the development process.
- Documentation: Serves as a basis for creating test cases and other documentation.

16. Class Diagram and Generalization with Examples

a. Class Diagram:

- A class diagram represents the static structure of a system by showing its classes, attributes, operations, and relationships.

Diagram:



b. Generalization:

- Generalization is the process of extracting shared characteristics from two or more classes and combining them into a generalized superclass.

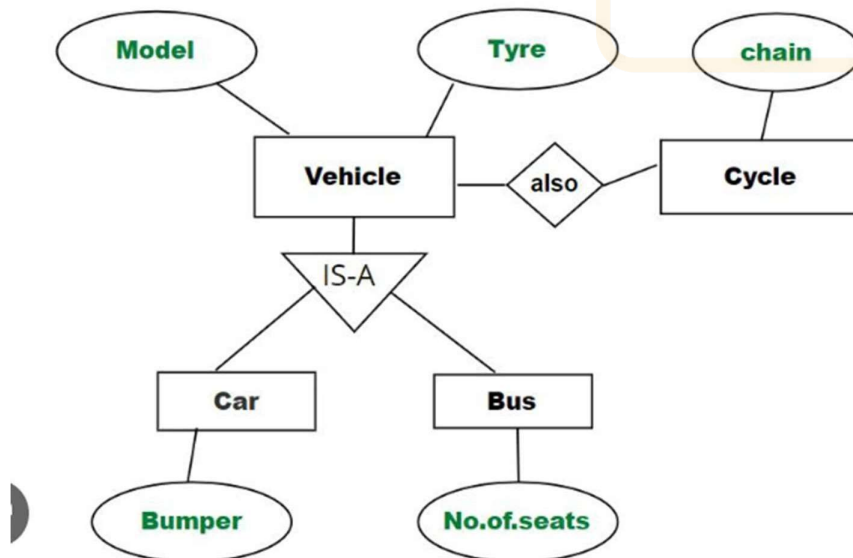
- Example:

- Superclass: Vehicle

- Subclasses: Car, Bike

- Shared Attributes: Make, Model, Tyre etc

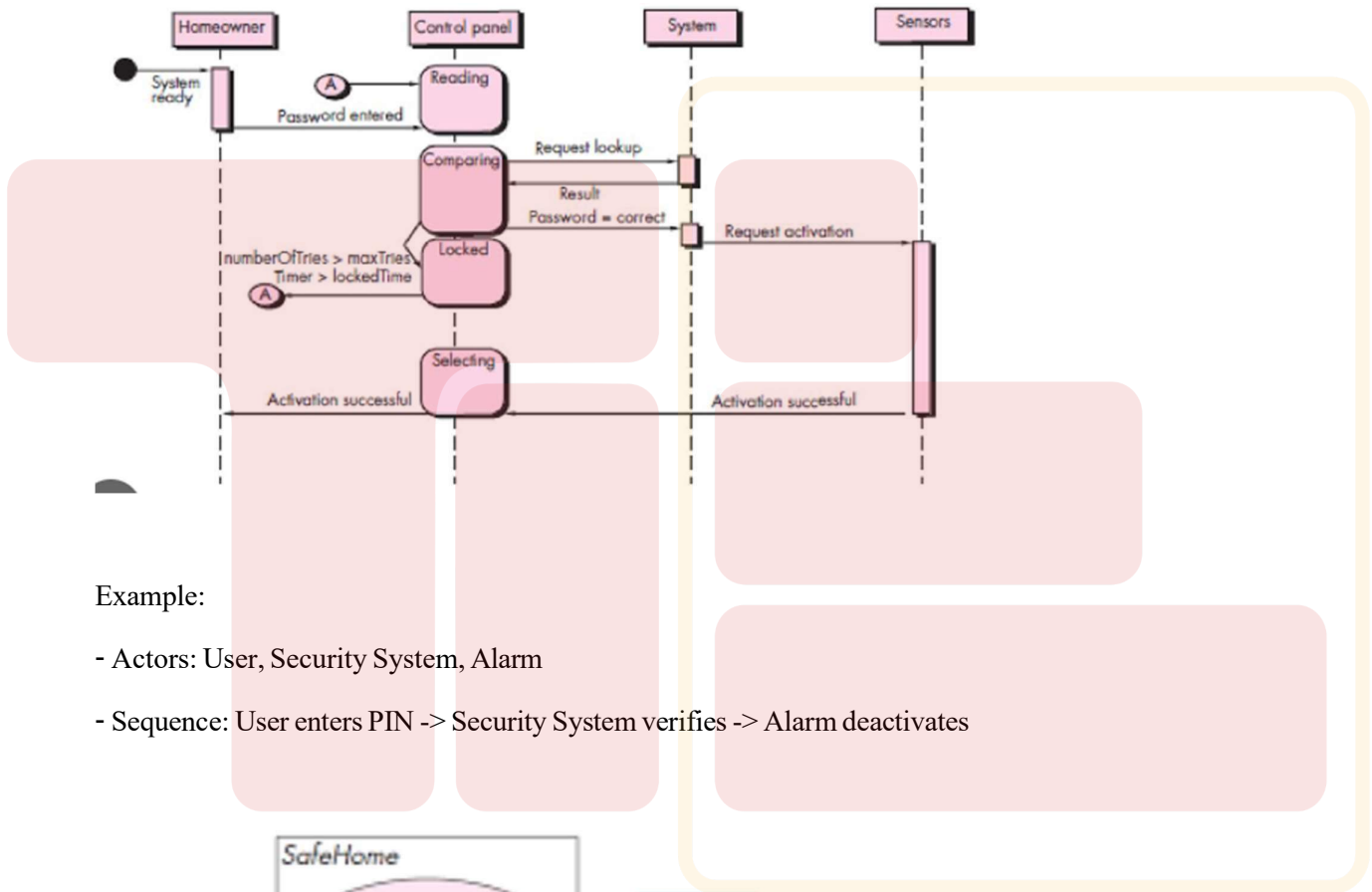
- Shared Operations: Start(), Stop()



17. Sequence and UML Diagram for Safe Home Security Function

- Sequence Diagram:

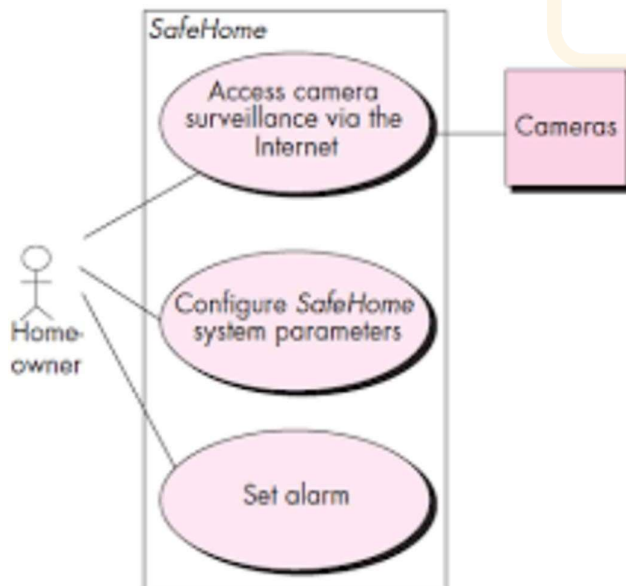
- Shows the interaction between objects over time.



Example:

- Actors: User, Security System, Alarm

- Sequence: User enters PIN -> Security System verifies -> Alarm deactivates



- UML Diagram:

- Represents the structure and behavior of the SafeHome system.
- Includes class diagrams, use case diagrams, and interaction diagrams.

Example:

- Classes: User, SecuritySystem, Alarm
- Relationships: User interacts with Security System; Security System controls Alarm

18. Activity Diagram for an ATM

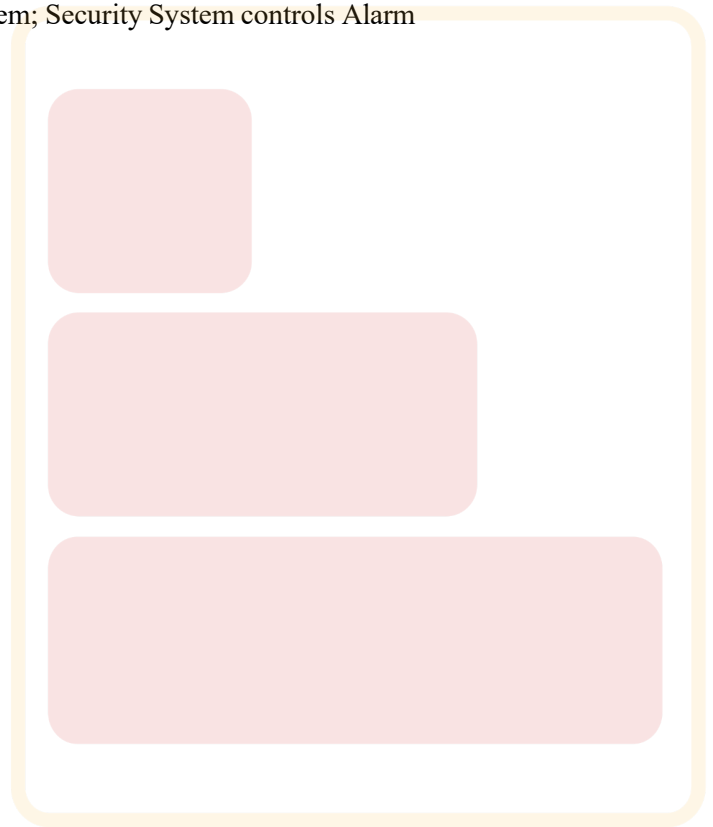
Activity Diagram:

- Describes the flow of activities in a system.

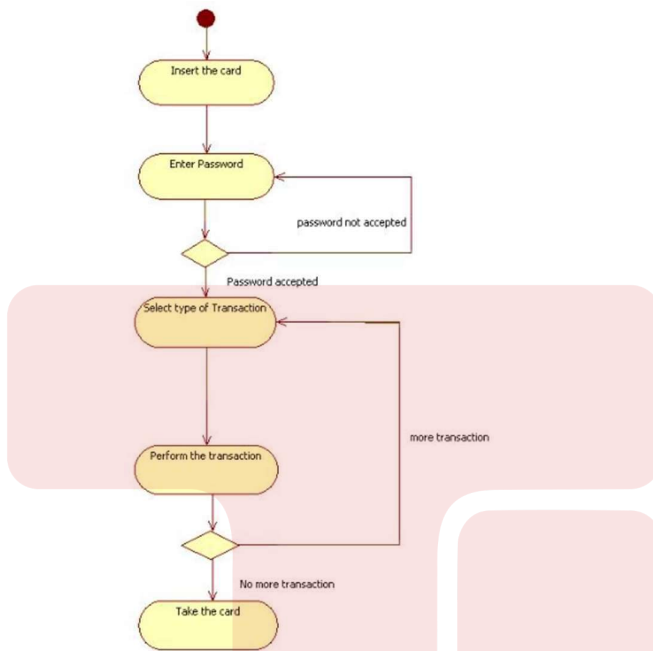
Example: ATM Withdrawal Process

Steps:

1. User inserts card.
2. ATM prompts for PIN.
3. User enters PIN.
4. ATM verifies PIN.
5. User selects withdrawal.
6. User enters amount.
7. ATM dispenses cash.
8. User takes cash.
9. ATM prints receipt.
10. User takes card and receipt.



Sample Diagram:



19. What is an Agile Process? Write the Principles to Achieve Agility.

Agile Process:

Agile is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, and customer satisfaction. Agile methodologies prioritize delivering small, workable pieces of software quickly and efficiently, with continuous feedback and adaptation throughout the development process.

Principles to Achieve Agility:

1. Customer Satisfaction:

- Deliver valuable software to customers early and continuously.

2. Welcome Changing Requirements:

- Embrace changes, even late in development, to provide a competitive advantage to the customer.

3. Deliver Working Software Frequently:

- Deliver working software in short, iterative cycles, ranging from a few weeks to a couple of months.

4. Collaboration:

- Developers and business stakeholders must work together daily throughout the project.

5. Motivated Individuals:

- Build projects around motivated individuals and provide them with the environment and support they need.

6. Face-to-Face Communication:

- The most effective way to convey information is through face-to-face communication.

7. Working Software:

- The primary measure of progress is working software.

8. Sustainable Development:

- Maintain a constant pace of development that is sustainable indefinitely.

9. Technical Excellence and Good Design:

- Continuous attention to technical excellence and good design enhances agility.

10. Simplicity:

- Maximize the amount of work not done, which is essential.

11. Self-Organizing Teams:

- The best architectures, requirements, and designs emerge from self-organizing teams.

12. Reflect and Adjust:

- Regularly reflect on how to become more effective, then tune and adjust behavior accordingly.

20. Extreme Programming Process (XP) with Diagram

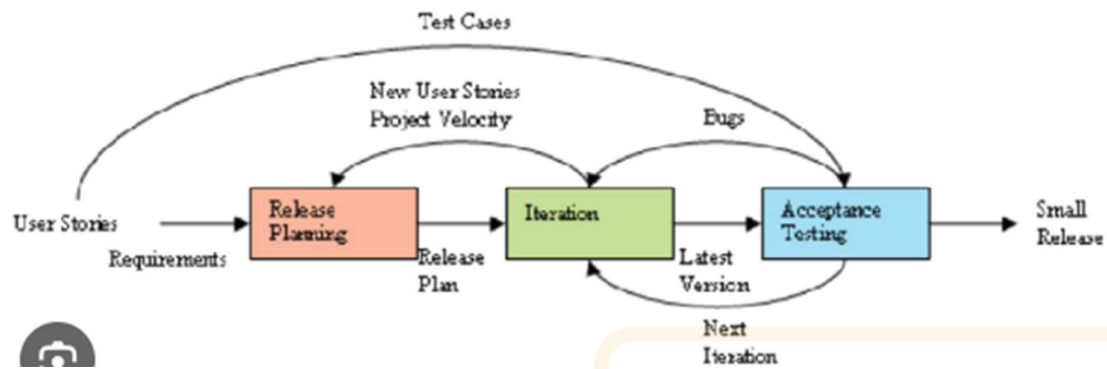
Extreme Programming (XP):

XP is an Agile software development methodology that emphasizes customer satisfaction, continuous feedback, and iterative development. XP promotes technical excellence and continuous improvement through practices like pair programming, test-driven development, and continuous integration.

Key Practices:

- Planning: Short, frequent releases with customer involvement.
- Design: Simple designs that meet current requirements.
- Coding: Emphasizes collective code ownership, pair programming, and refactoring.
- Testing: Test-driven development (TDD) ensures code quality and reliability.
- Continuous Integration: Regularly integrate and test code to detect issues early.

XP Process Diagram:



- User Stories: Define requirements.
- Iteration Planning: Plan the tasks for the iteration.
- Coding C Unit Testing: Develop and test code.
- Continuous Integration: Regularly integrate code.
- Refactoring: Improve code without changing functionality.
- Release: Deliver working software to the customer.
- Feedback Loop: Gather feedback for the next iteration.

21. Scrum Process Flow with Diagram

Scrum:

Scrum is an Agile framework that enables teams to work together to deliver a product in iterative cycles called sprints. Scrum emphasizes collaboration, accountability, and iterative progress towards a well-defined goal.

Scrum Process Flow:

1. Product Backlog:

- A prioritized list of features and requirements.

2. Sprint Planning:

- Select items from the product backlog to work on during the sprint.

3. Sprint:

- A time-boxed iteration (typically 2-4 weeks) where the team develops potentially shippable product increments.

4. Daily Scrum:

- A daily stand-up meeting to discuss progress, obstacles, and plans for the day.

5. Sprint Review:

- At the end of the sprint, the team presents the completed work to stakeholders for feedback.

6. Sprint Retrospective:

- The team reflects on the sprint and identifies ways to improve future sprints.

Scrum Process Diagram(sample):



- Product Backlog: Features and requirements.
- Sprint Planning: Select tasks for the sprint.
- Sprint: Develop and test features.

- Daily Scrum: Daily coordination meeting.
- Sprint Review: Review work with stakeholders.
- Sprint Retrospective: Reflect on the sprint.
- Product Increment: Deliverable software.

22. Templates for Defining a Feature and Feature-Driven Development Process (FDD)

Templates for Defining a Feature:

- Feature Title: A concise name for the feature.
- Description: A brief description of what the feature does.
- Priority: The importance of the feature (e.g., high, medium, low).
- Acceptance Criteria: Conditions that must be met for the feature to be considered complete.
- Dependencies: Other features or tasks that this feature depends on.
- Assumptions: Assumptions made when defining the feature.
- Risks: Potential risks associated with the feature.

Feature-Driven Development (FDD) Process:

FDD is an iterative and incremental software development methodology that focuses on building and designing features.

Steps in FDD:

1. Develop an Overall Model:
 - Create a high-level model of the system.
2. Build a Feature List:
 - Identify and list the features to be developed.
3. Plan by Feature:
 - Prioritize and plan the development of features.

4. Design by Feature:

- Create detailed designs for each feature.

5. Build by Feature:

- Develop, test, and integrate each feature.

- Develop Overall Model: High-level system design.

- Build Feature List: List all features.

- Plan by Feature: Prioritize and plan features.

- Design by Feature: Detailed feature design.

- Build by Feature: Develop and test features.

23. Short Notes

i) Principle that Guides Each Frame and Activity:

- Principles:

- Value: Focus on delivering value to customers.
- Feedback: Use continuous feedback to guide development.
- Simplicity: Simplify processes to reduce complexity.
- Collaboration: Encourage teamwork and communication.
- Iteration: Develop in small, manageable increments.

ii) RDMP (Risk-Driven Model Process):

- RDMP:

- A process that prioritizes risk management in software development. It involves identifying, analyzing, and mitigating risks throughout the project lifecycle.

- Steps:

1. Identify Risks
2. Analyze Risks
3. Plan Risk Mitigation
4. Implement Risk Mitigation
5. Monitor and Review Risks

24. Prototyping Model and Its Impact on Project Cost

Prototyping Model:

The prototyping model is a software development approach where a prototype (a preliminary version of the software) is built, tested, and refined based on user feedback. This process continues until a satisfactory prototype is developed, which then serves as the basis for the final product.

Effect on Overall Cost:

- Advantages:

- Reduced Risk: Early identification of issues and misunderstandings.
- Improved User Involvement: Continuous feedback from users ensures the final product meets their needs.
- Better Requirements: Helps in refining and clarifying requirements.

- Disadvantages:

- Increased Initial Costs: Developing and refining prototypes can be time-consuming and costly.
- Scope Creep: Users may request additional features, leading to scope expansion and increased costs.

25. Top-Down vs. Bottom-Up Design Strategies

Top-Down Design:

- Approach: Starts with a high-level design and breaks it down into smaller, more detailed components.
- Advantages:
 - Clear vision of the overall system.
 - Easier to manage and plan.
- Disadvantages:
 - May overlook low-level details early in the process.

Bottom-Up Design:

- Approach: Starts with detailed components and integrates them into higher-level structures.
- Advantages:
 - Focuses on implementing specific functionalities first.
 - More adaptable to changes.
- Disadvantages:
 - Can lead to integration challenges.
 - The overall system design may be less clear initially.

26. Definition and Characteristics of a Project

Definition of a Project:

A project is a temporary endeavor undertaken to create a unique product, service, or result. It has a defined beginning and end, specific objectives, and usually involves multiple stakeholders.

Characteristics of Projects:

1. Temporary: Projects have a defined start and end date.
2. Unique Output: Projects produce unique deliverables, products, or services.

3. Specific Objectives: Projects are driven by specific goals and objectives.
4. Resource Constraints: Projects are constrained by resources such as time, budget, and manpower.
5. Uncertainty: Projects involve risks and uncertainties.
6. Stakeholder Involvement: Projects require collaboration between various stakeholders.
7. Progressive Elaboration: Projects are developed in iterative steps with increasing detail.

27. Types of Requirements and ISO 12207 Software Development Life Cycle

Types of Requirements:

- Functional Requirements: Describe what the system should do (e.g., features, functionality).
- Non-Functional Requirements: Define system attributes like performance, security, and usability.
- Business Requirements: High-level needs of the organization (e.g., business goals).
- User Requirements: Specific needs of the end-users (e.g., user interface).
- System Requirements: Detailed descriptions of the system's functionality and performance.

ISO 12207 Software Development Life Cycle:

- ISO 12207: An international standard for software life cycle processes, providing a framework for software development and maintenance.

Phases:

1. System Requirements Analysis: Define system-level requirements.
2. Software Requirements Analysis: Define software-specific requirements.
3. Architectural Design: Define the software architecture.
4. Detailed Design: Develop detailed software design.

5. Implementation: Code and integrate the software.
6. Testing: Verify and validate the software.
7. Installation and Checkout: Deploy the software to the user environment.
8. Operation and Maintenance: Operate and maintain the software.

ISO 12207 Diagram-VTU PAPER 2004:

INSERT DIAGRAM HERE- RNSIT ARCHIEVES

28. Functions of the Object-Driven Development Process

Object-Driven Development (ODD):

ODD is a software development methodology that emphasizes the use of objects and object-oriented principles in design and development.

Functions:

1. Encapsulation: Group related data and behavior into objects, promoting modularity.
2. Abstraction: Hide implementation details and expose only essential features.
3. Inheritance: Reuse existing code by creating new classes that inherit from existing ones.
4. Polymorphism: Allow objects of different classes to be treated as objects of a common superclass.
5. Modularity: Break down the system into smaller, manageable components (objects).
6. Reusability: Promote code reuse through inheritance and modular design.
7. Maintainability: Simplify maintenance by organizing code into well-defined objects.

29. Test-Driven Development (TDD) Process and Benefits

Test-Driven Development (TDD):

TDD is a software development process where tests are written before the actual code is developed. The development cycle involves writing a test, writing code to pass the test, and then refactoring the code.

TDD Process:

1. Write a Test: Write a test case for a new function or feature.
2. Run the Test: Run the test to see it fail (since the function is not yet implemented).
3. Write Code: Write the minimum amount of code to pass the test.
4. Run the Test Again: Verify that the test now passes.
5. Refactor: Improve the code while ensuring that the test still passes.
6. Repeat: Continue the cycle for each new feature or function.

Benefits of TDD:

1. Improved Code Quality: Ensures that the code meets the requirements.
2. Fewer Bugs: Early detection of issues reduces the likelihood of bugs.
3. Better Design: Encourages simple and maintainable code.
4. Continuous Feedback: Provides immediate feedback on the code's correctness.
5. Documentation: Tests serve as documentation for the code's behavior.
6. Confidence in Refactoring: Safe refactoring since tests verify that functionality is preserved.

30. Definition of Management and Key Activities of Software Project Management

Management:

Management involves planning, organizing, leading, and controlling resources to achieve organizational goals. In software project management, it refers to the application of knowledge, skills, tools, and techniques to project activities to meet project requirements.

Key Activities of Software Project Management:

1. Project Planning:

- Define project scope, objectives, and deliverables.
- Create a project plan with timelines, resources, and milestones.

2. Resource Management:

- Allocate and manage resources (people, tools, budget) effectively.

3. Risk Management:

- Identify, analyze, and mitigate potential risks.

4. Quality Management:

- Ensure that the project meets quality standards and requirements.

5. Communication Management:

- Facilitate communication between stakeholders, team members, and management.

6. Time Management:

- Monitor project timelines and ensure that tasks are completed on schedule.

7. Cost Management:

- Control project costs to stay within the budget.

8. Stakeholder Management:

- Engage and manage relationships with stakeholders.

9. Change Management:

- Handle changes in project scope, requirements, or resources efficiently.

10. Project Monitoring and Control:

- Track project progress, measure performance, and take corrective actions as needed.

31. Importance of Project Planning and Principles of Project Management Processes

Importance of Project Planning:

Project planning is crucial in ensuring the success of a project. It provides a roadmap for the project team, outlining how the project will be executed, monitored, and controlled. Proper planning helps in identifying project goals, defining roles and responsibilities, estimating resources, and setting deadlines. It also helps in risk management, communication, and ensures that the project stays on track and within budget.

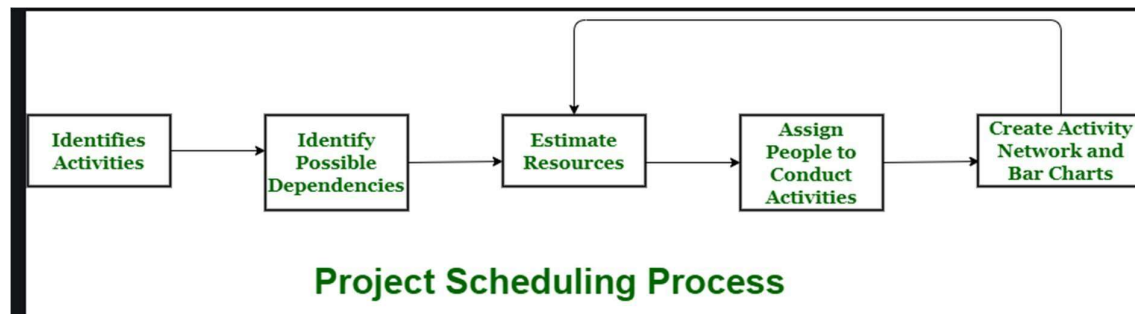
Principles of Project Management Processes:

1. Initiation: Define the project at a high level and secure approval.
2. Planning: Establish the project scope, objectives, and procedures.
3. Execution: Coordinate resources and execute the project plan.
4. Monitoring and Controlling: Track progress, manage changes, and ensure project objectives are met.
5. Closing: Finalize all project activities and formally close the project.

Project Scheduling Process:

Project scheduling is a key component of project planning, involving the identification of project tasks, their dependencies, and the time required to complete them.

Project Scheduling Diagram:



32. Project Control Cycle with Diagram

Project Control Cycle:

The project control cycle is a continuous process of monitoring, evaluating, and taking corrective action to ensure that the project stays on track and meets its objectives.

Steps in Project Control Cycle:

1. Set Performance Standards:

- Establish performance metrics and benchmarks.

2. Measure Performance:

- Monitor project progress and measure against the standards.

3. Compare Performance:

- Compare actual performance with planned performance.

4. Take Corrective Action:

- Implement changes to address deviations from the plan.

5. Reassess:

- Review the effectiveness of corrective actions and update plans as necessary.

33. Software Quality and Its Attributes, Process-Based Quality of Software

Software Quality:

Software quality refers to the degree to which a software product meets the specified requirements and satisfies the needs of the users. Quality attributes ensure that the software performs as expected and is reliable, maintainable, and usable.

Quality Attributes:

1. **Functionality:** The software's ability to provide the required functions.
2. **Reliability:** The software's ability to perform consistently under specified conditions.
3. **Usability:** The ease with which users can interact with the software.
4. **Efficiency:** The software's ability to perform well without wasting resources.
5. **Maintainability:** The ease with which the software can be modified and updated.
6. **Portability:** The software's ability to operate in different environments.

Process-Based Quality of Software:

Process-based quality focuses on the procedures and practices followed during the software development process. Adhering to a well-defined process ensures that quality is built into the software from the beginning.

Key Aspects:

- **Process Documentation:** Clearly defined procedures and standards.
- **Quality Assurance (QA):** Activities that ensure the process is followed correctly.
- **Continuous Improvement:** Regularly refining processes based on feedback and lessons learned.

34. Software Evolution Process with Diagram

Software Evolution Process:

The software evolution process refers to the continuous development, modification, and adaptation of software to meet changing requirements, correct defects, or improve performance.

Stages of Software Evolution:

1. Initial Development:

- The software is developed based on initial requirements.

2. Evolution:

- The software is modified to meet new requirements or improve functionality.

3. Servicing:

- Routine maintenance is performed to fix bugs and ensure continued operation.

4. Phase-out:

- The software is gradually retired as it becomes obsolete or is replaced by new systems.

5. Retirement:

- The software is decommissioned and no longer used.

Software Evolution Process Diagram:

[Initial Development] --> [Evolution] --> [Servicing] --> [Phase-out] --> [Retirement]

35. Key Points of Step-Wise Framework with Diagram

Step-Wise Framework:

The step-wise framework is a structured approach to project management that breaks down the project into a series of well-defined steps, each with specific objectives and outcomes.

Key Points:

1. Identify Objectives:

- Define the goals and scope of the project.

2. Define Project Scope:

- Determine the boundaries and deliverables of the project.

3. Develop Project Plan:

- Create a detailed plan outlining tasks, resources, and timelines.

4. Allocate Resources:

- Assign resources (people, budget, tools) to tasks.

5. Execute Plan:

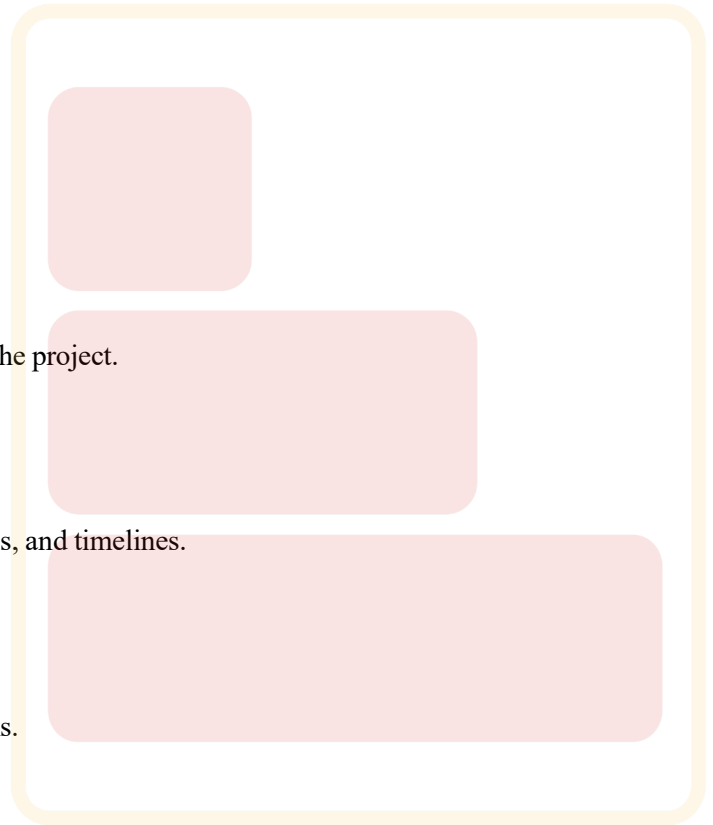
- Carry out the project according to the plan.

6. Monitor and Control:

- Track progress and make adjustments as needed.

7. Review and Close:

- Review project outcomes and formally close the project.



Step-Wise Framework Diagram:

[Identify Objectives] --> [Define Project Scope] --> [Develop Project Plan] --> [Allocate Resources] --> [Execute Plan] --> [Monitor and Control] --> [Review and Close]

36. Product and Process Metrics

Product Metrics:

Product metrics measure the attributes and characteristics of the software product itself.

Examples:

- Size Metrics: Lines of Code (LOC), Function Points (FP).
- Complexity Metrics: Cyclomatic complexity, Halstead metrics.
- Quality Metrics: Defect density, customer satisfaction.

Process Metrics:

Process metrics measure the efficiency and effectiveness of the software development process.

Examples:

- Effort Metrics: Person-hours, cost per phase.
- Productivity Metrics: Output per person-hour.
- Defect Metrics: Defect removal efficiency, defect leakage.

Importance:

- Product Metrics help in assessing the quality and performance of the software.
- Process Metrics help in improving the development process and ensuring that the project stays on track.

37. Test Closure and Test Automation with Diagram

Test Closure:

Test closure refers to the activities performed after testing is completed and before the project is delivered or released. It involves finalizing test documentation, archiving test artifacts, and evaluating the overall testing process.

Test Closure Activities:

1. Test Summary Report: Document the results of the testing phase.
2. Defect Closure: Ensure that all defects are resolved or documented.
3. Test Artifacts Archival: Store test cases, scripts, and results for future reference.
4. Evaluate Test Process: Analyse the testing process to identify areas for improvement.

Test Automation:

Test automation involves using software tools to execute test cases automatically. It is particularly useful for repetitive tasks and regression testing.

Test Automation Process:

1. Select Test Cases: Identify test cases suitable for automation.
2. Tool Selection: Choose the appropriate automation tool.
3. Script Development: Write scripts to automate test execution.
4. Execution: Run automated tests using the tool.
5. Result Analysis: Analyse the results and report any issues.

38. Different SEI CMM Levels

SEI CMM (Capability Maturity Model):

The SEI CMM is a framework developed by the Software Engineering Institute (SEI) that assesses the maturity of an organization's software development processes.

CMM Levels:

1. Level 1: Initial:

- Processes are unpredictable and poorly controlled. Success depends on individual efforts.

2. Level 2: Managed:

- Basic project management processes are established. Processes are repeatable for similar projects.

3. Level 3: Defined:

- Processes are well-documented and standardized across the organization.

4. Level 4: Quantitatively Managed:

- Processes are measured and controlled using quantitative metrics.

5. Level 5: Optimizing:

- Focus on continuous process improvement through feedback and innovation.

39. ISO 9001:2000 Process and Its Principles

ISO 9001:2000:

ISO 9001:2000 is an international standard for quality management systems (QMS). It provides a framework for organizations to ensure they meet customer and regulatory requirements and continuously improve their processes.

ISO 9001:2000 Principles:

1. Customer Focus:

- Understand and meet customer needs and strive to exceed customer expectations.

2. Leadership:

- Establish unity of purpose and direction. Leaders should create an environment that encourages employees to achieve quality objectives.

3. Involvement of People:

- Employees at all levels should be fully

engaged and empowered to contribute to quality improvement.

4. Process Approach:

- Manage activities and resources as processes to achieve efficient results.

5. System Approach to Management:

- Identify, understand, and manage interrelated processes as a system to improve the organization's effectiveness.

6. Continual Improvement:

- Focus on continuous improvement of processes and the quality management system.

7. Factual Approach to Decision Making:

- Make decisions based on the analysis of data and information.

8. Mutually Beneficial Supplier Relationships:

- Develop relationships with suppliers that benefit both parties and enhance the organization's ability to create value.

ISO 9001:2000 Process:

1. Plan: Establish objectives and processes necessary to deliver results according to customer requirements.
2. Do: Implement the processes.
3. Check: Monitor and measure processes against policies, objectives, and requirements, and report results.
4. Act: Take actions to continually improve process performance.